Chapter 7

Second-Stage Ranking and Multi-Task Learning

7.1 The Case for a Heavy Ranker

In Chapter 6, we built a first-stage ranker using gradient boosted trees and DeepFM. These models are efficient enough to score hundreds of candidates within tight latency budgets, but their expressive capacity is limited. They cannot easily model complex feature interactions across many fields, capture sequential user behavior, or jointly optimize for multiple business objectives.

The second-stage ranker (often called the heavy ranker) operates on a smaller, already-filtered candidate set. This relaxed constraint allows it to use deeper architectures, richer feature interactions, and multi-task prediction heads. In Instagram Explore's four-stage pipeline, this is the stage that uses a Multi-Task Multi-Label (MTML) neural network to predict engagement probabilities for clicks, likes, saves, and negative signals like 'see less.'

This chapter covers five influential architectures for the heavy ranking stage: Deep & Cross Network V2 (DCN-V2) for explicit feature interactions, Deep Learning Recommendation Model (DLRM) for large-scale systems with massive embedding tables, Deep Interest Network (DIN) and Deep Interest Evolution Network (DIEN) for attention-based user history modeling, and Multi-gate Mixture-of-Experts (MMoE) for multi-task learning. The accompanying code tutorials implement DCN-V2, DLRM, and MMoE on the Yambda dataset; DIN and DIEN are covered conceptually as important reference architectures for teams with rich user behavior sequences. Each model represents a different design philosophy for the same core problem: how to combine sparse categorical features, dense numerical features, and multiple optimization objectives into a single, powerful ranking model.

7.2 Modeling Feature Interactions at Depth

The central challenge in ranking is learning which feature combinations matter. A user's age alone is weakly predictive; combine it with the item's genre and the time of day, and you may discover a strong signal. The models in this chapter differ primarily in how they discover and represent these cross-feature patterns.

7.2.1 Evolution from Wide & Deep to DCN

Google's Wide & Deep model (2016) demonstrated the value of combining a linear component (for memorization of specific feature co-occurrences) with a deep neural network (for generalization). However, the 'wide' component required manually engineered feature crosses, which was labor-intensive and brittle.

The original Deep & Cross Network (DCN, 2017) addressed this by introducing a Cross Network that automatically learns explicit feature interactions of bounded degree. Each cross layer takes an input vector and produces a feature-crossed version using a learned weight vector. After L cross layers, the model can capture interactions up to order L+1.

DCN-V2 (Wang et al., 2021) generalized this further by replacing the weight vector with a full weight matrix, factorized using low-rank decomposition for efficiency.

7.3 Deep & Cross Network V2 (DCN-V2)

7.3.1 Architecture

DCN-V2 has two parallel pathways that process the same concatenated input (sparse feature embeddings + dense features):

- **Cross Network:** A sequence of cross layers where each layer computes: $x_{l+1} = x_0 * (W_l * x_l + b_l) + x_l$. The key innovation is that W_l is a full matrix (not a vector as in DCN-V1), factorized as $W = U * V^T$ for parameter efficiency.
- **Deep Network:** A standard multi-layer perceptron that captures implicit higher-order interactions through nonlinear activations.

The two pathways can be combined in parallel (concatenate outputs) or stacked (cross network feeds into deep network). The parallel structure is more common in practice.

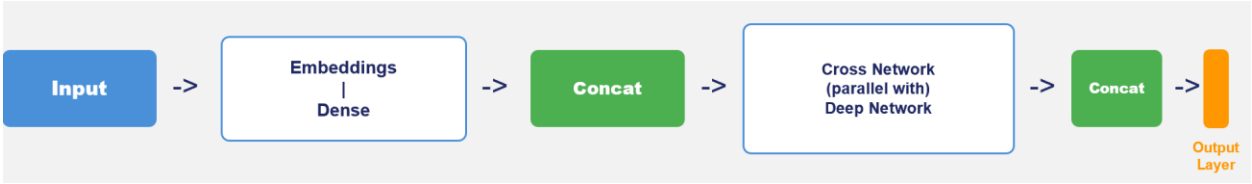

Figure 1: DCN-V2 architecture - Input -> [Embeddings | Dense] -> Concat -> Cross Network (parallel with) Deep Network -> Concat -> Output Layer

7.3.2 Low-Rank Cross Layers

The full weight matrix W in each cross layer has $O(d^2)$ parameters where d is the input dimension. For large feature spaces, this becomes prohibitive. DCN-V2's low-rank factorization $W = U * V^T$ reduces this to $O(d * r)$ where r is the rank, typically $r << d$.

This maintains the expressiveness of full-matrix crosses while keeping the model tractable.

```
Cross layer: x_{l+1} = x_0 * (U * V^T * x_l + b_l) + x_l
```

7.3.3 Design Choices

- **Number of cross layers:** Determines the maximum order of explicit feature interactions. 2-3 layers are typical for recommendation tasks.
- **Rank of W:** Controls the tradeoff between expressiveness and parameter count. Start with rank = d/4 and tune.
- **Parallel vs. stacked:** Parallel is generally preferred in practice for its stability and because both pathways can be computed concurrently at inference time.

Author's note: Google published practical lessons from deploying DCN at web scale (Wang et al., 2021). They found that the low-rank factorization not only reduced parameters but sometimes improved generalization by acting as a regularizer.

```
# DCN-V2 cross layer (from models/dcn.py):
class CrossNetworkV2(nn.Module):
    def forward(self, x0: torch.Tensor) -> torch.Tensor:
        xl = x0
        for i in range(self.num_layers):
            # Low-rank: W = U * V^T
            xl_w = self.V[i](self.U[i](xl))  # (batch, d)
            xl = x0 * (xl_w + self.bias[i]) + xl  # residual
        return xl
```

7.4 Deep Learning Recommendation Model (DLRM)

7.4.1 Architecture

Facebook's DLRM (Naumov et al., 2019) takes a distinctive approach to feature interactions. Rather than learning arbitrary cross layers, it explicitly computes pairwise dot products between all feature embeddings, then processes these interactions through a top-level MLP.

The architecture has four components:

- **Embedding tables:** Each sparse categorical feature gets its own embedding table. This is where the bulk of the model's parameters reside.
- **Bottom MLP:** Dense numerical features are projected through an MLP to the same dimensionality as the sparse embeddings. This is critical: it places dense and sparse features in the same vector space.

- **Interaction layer:** All embeddings (from sparse features and the bottom MLP output) are combined via pairwise dot products. This produces a vector of interaction terms.
- **Top MLP:** The concatenation of the interaction terms and the bottom MLP output is passed through a final MLP to produce the prediction.

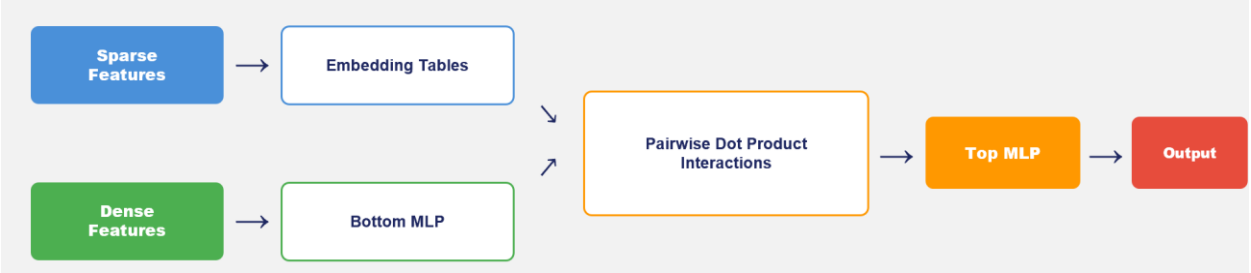

Figure 2: DLRM architecture - Sparse Features -> Embedding Tables and Dense Features -> Bottom MLP, both feeding into Pairwise Dot Product Interactions -> Top MLP -> Output

7.4.2 Design Philosophy

DLRM's design philosophy is 'interactions are all you need.' By computing explicit pairwise interactions between all feature embeddings, it captures the most important second-order signals without the overhead of learning arbitrary cross transformations.

- The bottom MLP is key: it transforms dense features so they are comparable to sparse embeddings in the dot product space
- The pairwise dot products scale as O(n^2) with the number of feature groups, but since the number of feature groups is typically small (10-30), this is manageable
- DLRM was designed for distributed training: embedding tables (which are memory-bound) can be sharded across machines, while MLPs (which are compute-bound) run on GPUs

Understanding the distinction between memory-bound and compute-bound operations is important for reasoning about system performance during serving. A memory-bound operation spends most of its time waiting for data to be fetched from memory rather than performing arithmetic. Embedding table lookups are the canonical example: for each sparse feature, the model must read a row from a potentially massive table stored in memory. The computation itself (a simple table lookup) is trivial, but the memory access pattern is irregular and cannot be easily parallelized or cached. When your model has hundreds of millions of embedding parameters, these lookups become the bottleneck.

A compute-bound operation, by contrast, spends most of its time performing arithmetic. MLP forward passes are compute-bound: they involve dense matrix multiplications that

modern GPUs can execute extremely efficiently using their thousands of parallel cores. DLRM's architecture is designed around this distinction: it places embedding tables (memory-bound) on CPU or distributed parameter servers where memory is abundant, and MLP computations (compute-bound) on GPUs where arithmetic throughput is highest. This hybrid placement strategy is why DLRM scales to the massive embedding tables used at Meta's scale.

Sample code snippet is given below.

```
# DLRM dot-product interactions (from models/dlrm.py):
class DotInteraction(nn.Module):
    def forward(self, embeddings: torch.Tensor):
        # embeddings: (batch, num_features, embed_dim)
        dots = torch.bmm(embeddings, embeddings.transpose(1, 2))
        # Extract upper triangle (pairwise interactions)
        idx = torch.triu_indices(n, n, offset=1)
        return dots[:, idx[0], idx[1]]
```

7.4.3 When to Choose DLRM vs. DCN-V2

Aspect	DCN-V2	DLRM
Interaction modeling	Learned cross layers (any order)	Explicit pairwise dot products
Parameter efficiency	Low-rank factorization	Compact (dot products are parameter-free)
Dense feature handling	Concatenated with embeddings	Projected via Bottom MLP
Distributed training	Standard data parallelism	Model parallelism for embeddings
Best for	Rich cross-feature discovery	Large-scale systems with many sparse features

7.5 Attention-Based Ranking: DIN and DIEN

While DCN-V2 and DLRM model feature interactions in a static manner, many recommendation scenarios require modeling how a user's interests relate to the specific candidate being scored. Alibaba's Deep Interest Network (DIN) and its successor DIEN introduced attention mechanisms to address this.

7.5.1 DIN: Target-Aware Attention over User History

DIN (Zhou et al., 2018) treats the user's historical behavior as a sequence of item embeddings and applies an attention mechanism to weight past items by their

relevance to the current candidate. This produces a dynamic user representation that changes depending on which item is being scored.

- A user who browsed both electronics and books will have different 'active interests' when scoring a laptop vs. a novel
- The attention weight between a historical item and the candidate is computed via a small network that takes both embeddings as input
- Complexity is O(T) per candidate where T is the history length, which is lightweight

Sample code snippet is given below.

```
# DIN attention mechanism (conceptual):
def din_attention(history_embeds, target_embed):
    """Compute attention weights over user history."""
    # history_embeds: (batch, seq_len, dim)
    # target_embed: (batch, dim)
    target = target_embed.unsqueeze(1)  # (batch, 1, dim)
    # Attention via element-wise product + MLP
    cross = history_embeds * target
    attn_input = torch.cat(
        [history_embeds, target.expand_as(history_embeds), cross],
        dim=-1
    )
    weights = F.softmax(attn_mlp(attn_input), dim=1)
    user_interest = (history_embeds * weights).sum(dim=1)
    return user_interest  # (batch, dim)
```

7.5.2 DIEN: Modeling Interest Evolution

DIEN (Zhou et al., 2019) extends DIN by modeling how user interests evolve over time using recurrent neural networks. It introduces two stages:

- **Interest Extractor:** A GRU that processes the user's behavior sequence chronologically, producing hidden states that represent the user's latent interest at each point in time.
- **Interest Evolution:** An Attentional Update Gate (AUGRU) that modulates the GRU's hidden state updates using attention scores relative to the target item. This focuses the sequential modeling on the subset of the user's evolving interests that are relevant to the candidate.

DIEN is more computationally expensive than DIN (two sequential passes over the history) but captures temporal dynamics that static attention cannot. Alibaba reported a 20.7% CTR improvement from DIEN in their Taobao ad system.

7.6 Multi-Task Learning for Ranking

Production recommender systems rarely optimize for a single metric. A music platform cares about whether the user listens (engagement), completes the track (satisfaction), likes it (explicit positive signal), or dislikes it (negative signal). A multi-task model predicts all of these simultaneously, sharing learned representations across tasks while allowing each task its own specialized prediction head.

7.6.1 Why Multi-Task Learning

- **Data efficiency:** Sparse tasks (likes, at ~4-8% positive rate) benefit from shared representations learned on abundant tasks (listens). Without multi-task learning, a like-prediction model would struggle with the limited positive signal.
- **Representation quality:** Learning to predict multiple outcomes forces the shared representation to capture richer, more generalizable patterns than any single-task model.
- **Serving efficiency:** A single forward pass produces all task predictions, reducing inference latency compared to running separate models for each objective.
- **Value function input:** Multi-task predictions provide the raw scores that the value function (Chapter 8) combines into a final ranking score.

7.6.2 The Challenge of Task Conflicts

Not all tasks benefit from shared representations. Tasks can conflict when optimizing for one hurts another. For example, optimizing aggressively for click-through rate might increase clickbait that leads to dislikes. A naive approach of simply summing task losses with shared parameters can lead to negative transfer, where the model performs worse on some tasks than dedicated single-task models would.

This is the central challenge that MMoE and related architectures address.

7.7 Multi-gate Mixture-of-Experts (MMoE)

7.7.1 Architecture

MMoE (Ma et al., KDD 2018) introduces a set of shared expert sub-networks and task-specific gating networks that learn how to route information from experts to tasks.

- **Experts:** A set of N expert networks (typically MLPs) that each process the full input. Each expert can specialize in different aspects of the feature space.
- **Gates:** Each task has its own gating network that produces a softmax distribution over the experts. This determines how much each expert contributes to that task's prediction.

- **Towers:** Each task has a task-specific tower (a small MLP) that takes the gated expert mixture as input and produces the final prediction.

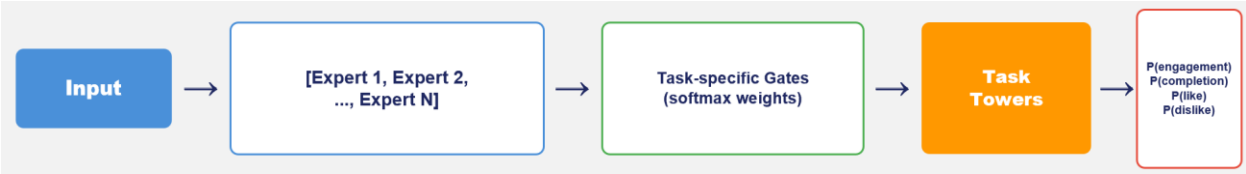

Figure 3: MMoE architecture - Input -> [Expert 1, Expert 2, ..., Expert N] -> Task-specific Gates (softmax weights) -> Task Towers -> [P(engagement), P(completion), P(like), P(dislike)]

The key insight is that the gates are learned end-to-end. If two tasks are synergistic, their gates will assign similar weights to the same experts. If tasks conflict, their gates will learn to rely on different experts, effectively partitioning the shared representation.

Sample code snippet is given below.

```
# MMoE gating mechanism (from models/mmoe.py):
expert_outs = torch.stack(
    [expert(x) for expert in self.experts], dim=1
)  # (batch, num_experts, expert_dim)

for task_idx, task_name in enumerate(self.task_names):
    gate_weights = F.softmax(
        self.gates[task_idx](x), dim=-1
    )  # (batch, num_experts)
    gated = torch.sum(
        expert_outs * gate_weights.unsqueeze(-1), dim=1
    )  # (batch, expert_dim)
    output[task_name] = self.towers[task_idx](gated)
```

7.7.2 Task Configuration

In our Yambda tutorial, we configure four prediction tasks:

Task	Type	Label Definition	Positive Rate	Loss
Engagement	Binary	played_ratio > 0	~95%	BCE
Completion	Binary	played_ratio >= 50%	~63%	BCE
Like	Binary	User liked item	~4-8%	Focal Loss
Dislike	Binary	User disliked item	~1-2%	Focal Loss

7.7.3 Handling Class Imbalance with Focal Loss

The like and dislike tasks are severely imbalanced: only 4-8% of interactions result in a like, and 1-2% in a dislike. Standard binary cross-entropy gives equal weight to all samples, which means the model sees predominantly negative examples and can learn a degenerate 'predict negative' strategy.

Focal Loss (Lin et al., 2017) addresses this by down-weighting well-classified (easy) examples and focusing the loss on hard, misclassified cases:

*FL(p_t) = -alpha_t * (1 - p_t)^gamma * log(p_t)*

where gamma controls how aggressively easy examples are down-weighted (gamma=0 recovers standard BCE, gamma=2 is a common starting point) and alpha_t balances the class weights. For our sparse tasks, focal loss allows the model to learn meaningful positive-class representations without being overwhelmed by the abundant negative class.

```
# Focal Loss (from models/mmoe.py):
class FocalLoss(nn.Module):
    def __init__(self, alpha=0.25, gamma=2.0):
        self.alpha, self.gamma = alpha, gamma

    def forward(self, pred, target):
        bce = F.binary_cross_entropy(
            pred, target, reduction="none"
        )
        p_t = pred * target + (1 - pred) * (1 - target)
        focal_weight = self.alpha * (1 - p_t) ** self.gamma
        return (focal_weight * bce).mean()
```

7.7.4 Loss Weighting and Training

The total loss is a weighted sum of individual task losses:

*L_total = w_eng * L_engagement + w_comp * L_completion + w_like * L_like + w_dis * L_dislike*

The weight hyperparameters (w_eng, w_comp, w_like, w_dis) control the relative importance of each task during training. Tasks with sparse positive labels (like, dislike) typically receive higher weights (e.g., 2.0x) to compensate for their lower gradient magnitude.

Loss weighting is as much art as science. Start with equal weights, monitor per-task metrics, and adjust weights for tasks that are underperforming. If a task's AUC plateaus while others improve, increase its weight. Automated approaches like uncertainty weighting exist but manual tuning is often sufficient for 3-5 tasks. The next chapter talks

in more detail about value functions that allow the various weights to be combined together.

7.8 Progressive Layered Extraction (PLE)

While MMoE allows flexible expert routing, all experts still share the same input and are trained jointly. When task conflicts are severe, the shared experts may compromise: they optimize for a middle ground that satisfies no task fully.

Progressive Layered Extraction (PLE, Tang et al., RecSys 2020) addresses this by introducing task-specific experts alongside shared experts, organized in progressive layers:

- **Shared experts:** Process input features to learn patterns useful across all tasks (same as MMoE).
- **Task-specific experts:** Each task gets its own set of expert networks that specialize exclusively for that task.
- **Progressive extraction:** Multiple levels of expert groups are stacked. Each level's gate can select from both shared and task-specific experts of the previous level, progressively separating shared and task-specific knowledge.

PLE provides stronger task isolation than MMoE, preventing negative transfer when tasks genuinely conflict. The cost is additional parameters (task-specific expert networks) and slightly more complex training. In practice, PLE is preferred over MMoE when you observe that certain tasks consistently degrade when trained jointly.

7.9 Practical Patterns for Heavy Ranking

7.9.1 Embedding Design

Embedding tables dominate the parameter count of heavy ranking models. Design choices that significantly affect model quality and serving cost:

- **Embedding dimension:** Larger dimensions capture richer representations but increase memory and computation. 16-32 dimensions per feature is typical; user and item IDs may use 64-128.
- **Sharing vs. separate embeddings:** All models in this chapter share embedding layers across the cross/deep pathways and across tasks. This is both efficient and beneficial for learning.
- **Pre-trained embeddings:** Item embeddings from Chapter 10 can be loaded as initializations, giving the ranking model a head start on item representations.

7.9.2 Number of Experts

For MMoE, a common heuristic is to use slightly more experts than tasks: for 4 tasks, start with 6 experts. This gives the model flexibility to allocate some experts to individual tasks and some to shared patterns. Too few experts force excessive sharing; too many create sparse gating distributions that are hard to train.

7.9.3 Calibration

Multi-task models produce predicted probabilities for each task, but these probabilities are often miscalibrated: the model may predict P(like) = 0.3 when the true positive rate among those predictions is only 0.15. Since the value function in Chapter 8 combines these probabilities with weights, calibration directly affects ranking quality.

- **Platt scaling:** Fit a logistic regression on the model's logits using a held-out calibration set. Simple and effective.
- **Temperature scaling:** Divide logits by a learned temperature parameter. Even simpler: a single parameter per task.
- **Reliability diagrams:** Visualize calibration by plotting predicted probability bins against actual positive rates. Well-calibrated models follow the diagonal.

7.10 Deployment Considerations

7.10.1 Latency Budget

The heavy ranker operates under a larger latency budget than the first stage (100-200ms vs. 50-100ms for the first stage) because it scores fewer candidates. Even so, compute-intensive operations like attention over long user histories or large embedding lookups need careful optimization.

- Batch all candidates for a user into a single forward pass
- Use mixed-precision inference (FP16) to halve memory bandwidth and double throughput on modern GPUs
- Pre-compute and cache user-side features that do not change between candidates
- Consider knowledge distillation: train the heavy ranker offline, then distill its knowledge into a lighter model for serving if latency is critical

7.10.2 Artifact Handoff to Chapter 8

The heavy ranker's multi-task predictions are the input to the value function (Chapter 8). The code tutorial exports two types of artifacts:

- **Full inference pipeline:** Model checkpoint, feature processors, and inference script for realistic production-like code.

- **Pre-computed predictions:** Parquet files with predicted probabilities and true labels for the test set, enabling rapid iteration on value function weights without re-running inference.

7.11 Code Tutorial: DCN-V2, DLRM, and MMoE on Yambda

The code tutorial for this chapter implements all three architectures on the same Yandex Yambda dataset used in Chapter 6, enabling direct comparison.

7.11.1 Data Pipeline

The tutorial uses a Polars-based data pipeline for feature engineering, taking advantage of Polars' lazy evaluation for memory-efficient processing of the 50 million interaction dataset. The same Global Temporal Split protocol from Chapter 6 is used for train/test separation.

7.11.2 Single-Task Models: DCN-V2 and DLRM

Both DCN-V2 and DLRM are trained on listen completion as a single-task binary classification. This allows direct comparison with Chapter 6's XGBoost and DeepFM baselines. The tutorial demonstrates how to configure the cross network rank for DCN-V2 and the bottom MLP dimensions for DLRM.

7.11.3 Multi-Task Model: MMoE

The MMoE implementation trains on four tasks simultaneously. Key implementation details include:

- Focal loss for the sparse like and dislike tasks to handle class imbalance
- Configurable task weights for the combined loss function
- Gate weight visualization to interpret which experts are used by which tasks
- Export of prediction artifacts (Parquet files and calibration data) for Chapter 8

7.12 Chapter Summary

- The second-stage (heavy) ranker operates on a filtered candidate set and can afford deeper, more expressive architectures
- DCN-V2 learns explicit feature interactions through cross layers with low-rank weight matrices, automating what was manual feature engineering in earlier approaches
- DLRM uses pairwise dot products between embeddings for interaction modeling, with a bottom MLP to project dense features into the embedding space
- DIN and DIEN introduce target-aware attention and sequential interest evolution for scenarios where user behavior history is a strong signal

- Multi-task learning (MMoE, PLE) enables a single model to predict multiple engagement signals, providing efficiency and representation benefits
- MMoE uses shared experts with task-specific gates to handle task conflicts; PLE adds task-specific experts for stronger isolation when conflicts are severe
- Focal loss is critical for severely imbalanced tasks like likes and dislikes
- Calibration of multi-task predictions is essential before passing them to the value function in Chapter 8

References

- Wang, R. et al. (2021). *DCN V2: Improved Deep & Cross Network and Practical Lessons for Web-scale Learning to Rank Systems.* WWW 2021.
- Naumov, M. et al. (2019). *Deep Learning Recommendation Model for Personalization and Recommendation Systems.* Facebook Research.
- Ma, J. et al. (2018). *Modeling Task Relationships in Multi-task Learning with Multi-gate Mixture-of-Experts.* KDD 2018.
- Tang, H. et al. (2020). *Progressive Layered Extraction (PLE): A Novel Multi-Task Learning Model for Personalized Recommendations.* RecSys 2020.
- Zhou, G. et al. (2018). *Deep Interest Network for Click-Through Rate Prediction.* KDD 2018.
- Zhou, G. et al. (2019). *Deep Interest Evolution Network for Click-Through Rate Prediction.* AAAI 2019.
- Lin, T.-Y. et al. (2017). *Focal Loss for Dense Object Detection.* ICCV 2017.
- Cheng, H.-T. et al. (2016). *Wide & Deep Learning for Recommender Systems.* DLRS Workshop, RecSys 2016.
- Meta Engineering. (2023). *Scaling Instagram Explore Recommendations.* engineering.fb.com.
- Guo, H. et al. (2017). *DeepFM: A Factorization-Machine based Neural Network for CTR Prediction.* IJCAI 2017.
- Yandex. (2025). *Yambda-5B: A Large-Scale Multi-modal Dataset for Ranking and Retrieval.* arXiv:2505.22238.